

DS3000/DS9000

**INTRO TO
MACHINE LEARNING**

Lecture 1

September 8, 2025

Outline

❑ Course Information

- Instructor Information
- Class Schedule
- Evaluation

❑ Machine Learning Landscape

❑ Intro to Linear Regression

Instructor Information

- ❑ Dr. Mathias Babin

- ❑ Instructor for CS4480, CS4482, and DS3000

- ❑ Contact:

- ❑ **Email:** mbabin2@uwo.ca

- ❑ Consultation hours:

- ❑ By appointment

- ❑ My research is focused on AI applications for procedural content generation in games:

- ❑ Procedural level generation

- ❑ AI-based anti-cheat systems for online games

- ❑ RL-based approaches to game Balance

- ❑ Optimizations for PCG algorithm.

Evaluation

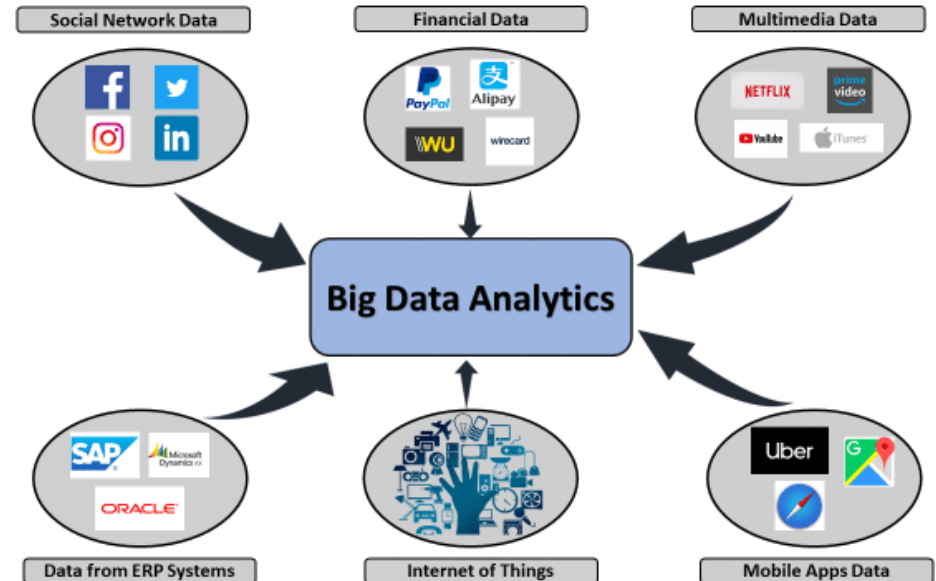
Assessment Type	Undergraduate %	Graduate %	Tentative Due Dates
3 Assignments	10% (total)	10% (total)	Sep 26, Oct 17, Nov 28
Group Project	35%	50%	Nov 14, Dec 5, Dec 9
Final Examination	55%	40%	TBA

1.1: Machine Learning Landscape

Big Data

- ❑ We are collecting and storing data at an unprecedented rate.

- Social media
- Online shopping
- Google Maps, Waze, Uber
- Medical data
- Satellite data
- IoT devices
- Chat and conversation
- Phone call records
- Video game and user actions



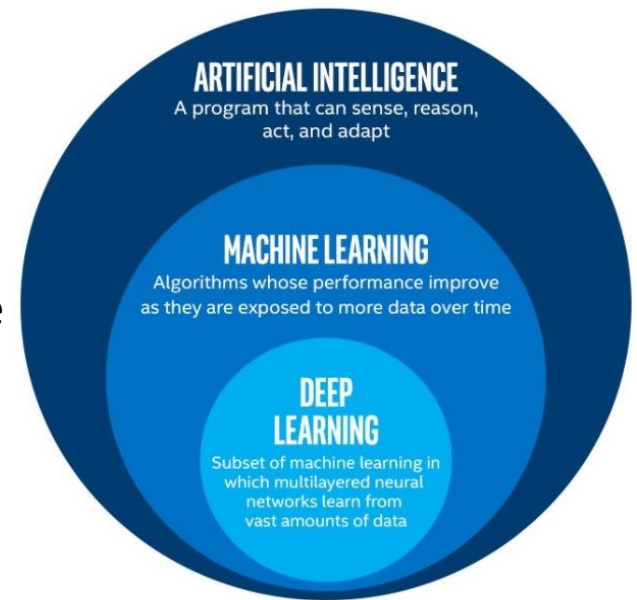
Big Data

- ❑ What do you do with all this data?
 - Too much data to search through it manually.
- ❑ But there is valuable information in the data.
 - How can we use it?

Artificial Intelligence

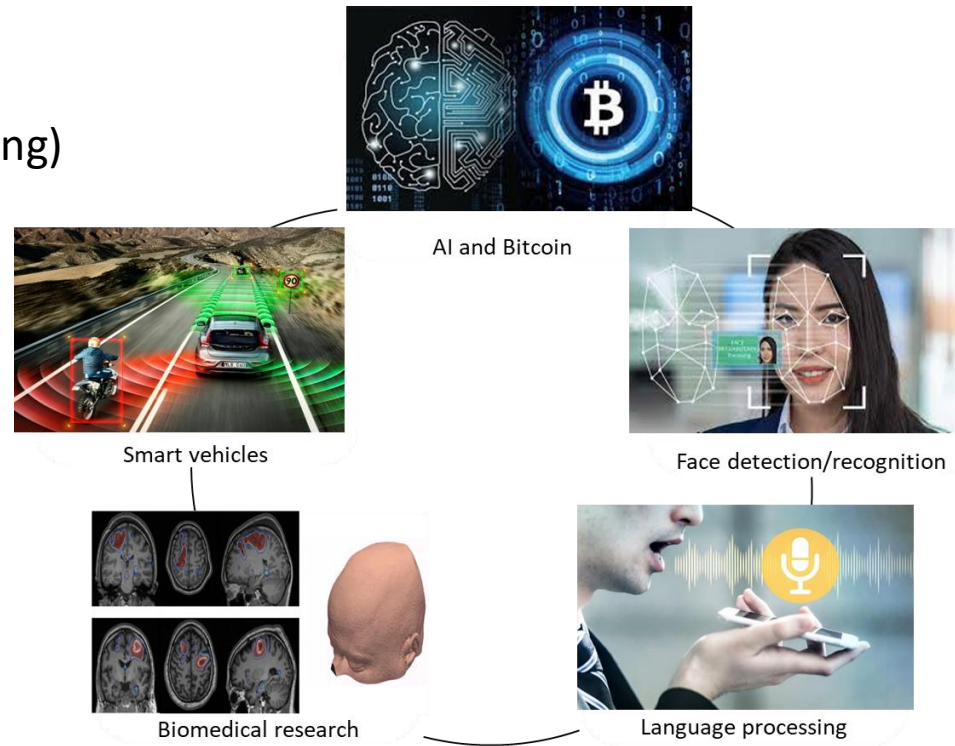
Artificial Intelligence

- ❑ Artificial intelligence
 - Born in the **1950s**
 - Scientists started asking whether computers could be made to “think”!
 - **AI: effort to automate intellectual tasks normally performed by humans.**
- ❑ Machine learning
 - Started to flourish in the 1990s.
 - By availability of **faster hardware and larger datasets**, ML quickly became the most popular and successful subfield of AI.
 - ML is using computers (mathematical statistics) to automatically detect patterns in data and use these to make predictions or decisions.
 - To automate something a human can do and can’t do (look at 1 TB of data)
- ❑ Deep learning
 - Is a subfield of ML and a new take on learning meaningful representations from data successively.
 - The “deep” isn’t deeper understanding; rather, stands for **successive layers of representations** (“layered representations learning” or “hierarchical representations learning”)
 - Powering modern tools like Instagram, TikTok, Instagram, Chatbots etc.



Applications of Machine Learning (ML)

- ❑ Machines can do tasks that typically require human intelligence, by learning from data/experience
 - ❑ ML can be applied in:
 - Autonomous vehicles
 - Healthcare (diagnosis, medical imaging)
 - Law enforcement
 - Finance
 - Natural language processing
 - Beating human in Go/Starcraft
 - Mimicking artistic styles
 - Animation
 - Image/Video from text
 - Cyber security
- Many more ...



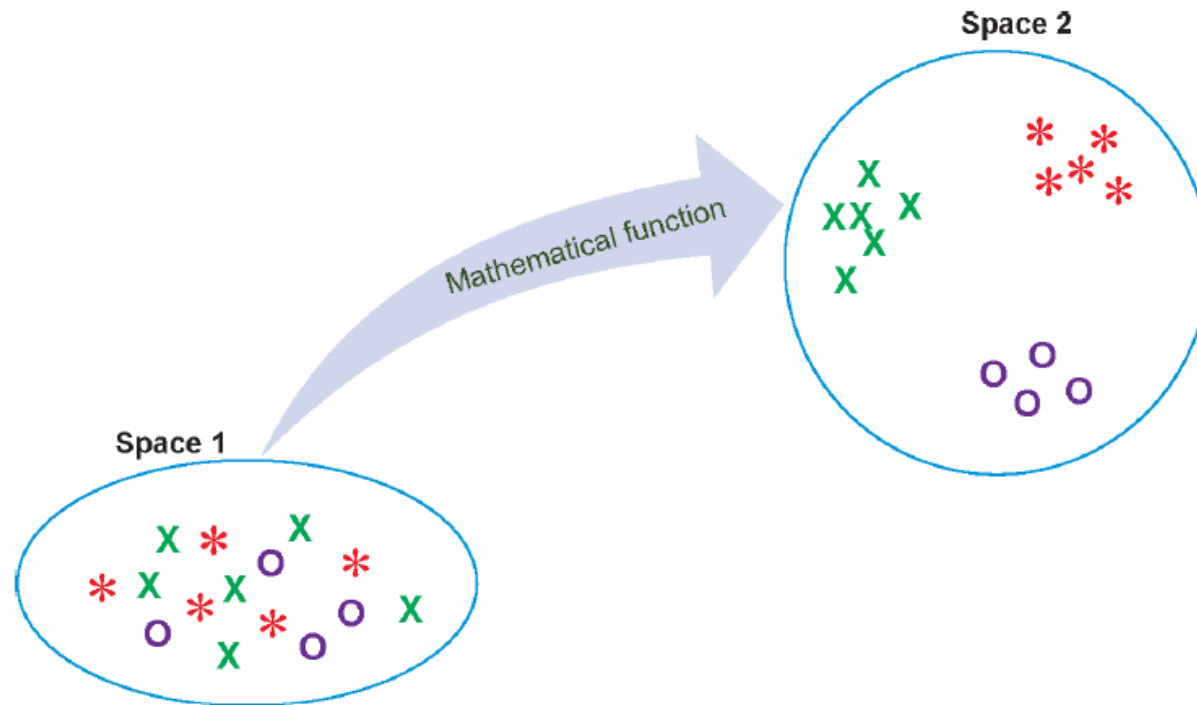
What is Machine Learning?

- ❑ ML discovers rules for executing a data processing task, given examples of what's expected (*Supervised learning*).
- ❑ To do *supervised* ML, we need three components:
 - Input data points
 - Examples of the expected output
 - Learning objective to measure whether the algorithm is doing a good job (distance between the algorithm's output and labelled example data).

ML is a meaningful input-to-target mapping: learning useful representations of the input data that get us closer to the expected output.

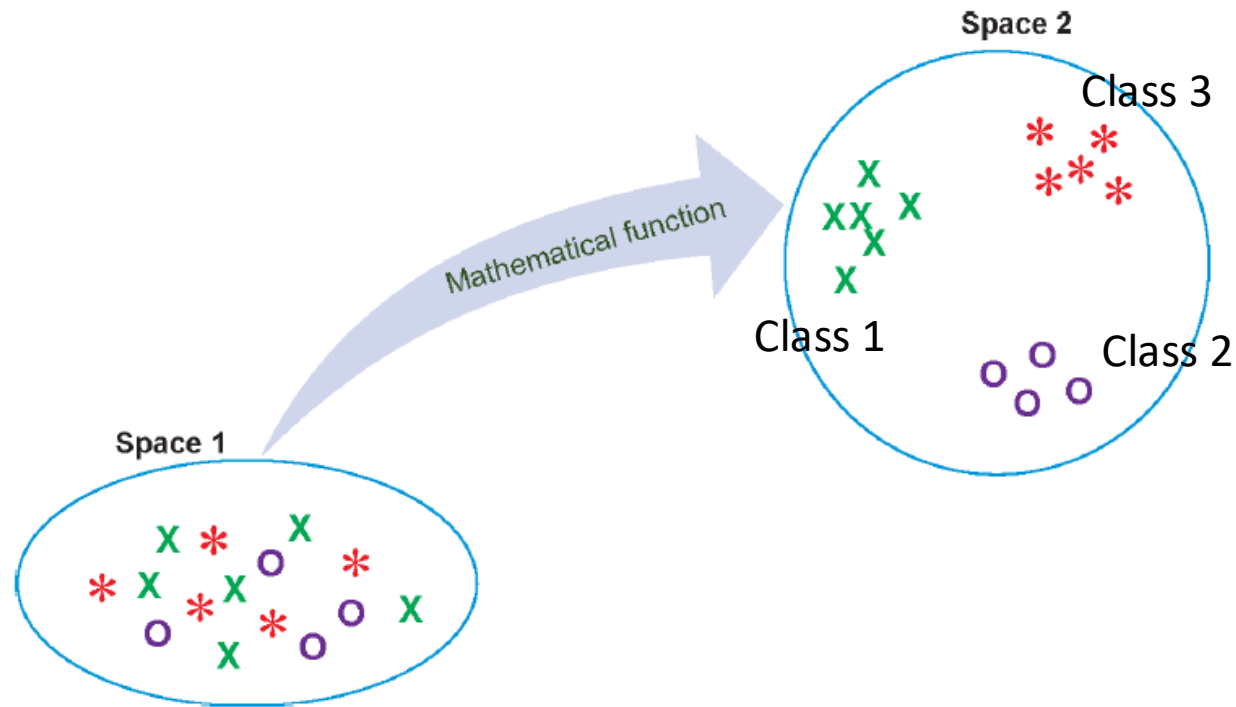
Mathematical View of Learning

- Learning process is indeed the process of finding parameters of an appropriate mathematical **mapping** by which an input vector is mapped to another space where similarities and dissimilarities of the “within-class” and “between class” data, are better projected, respectively.



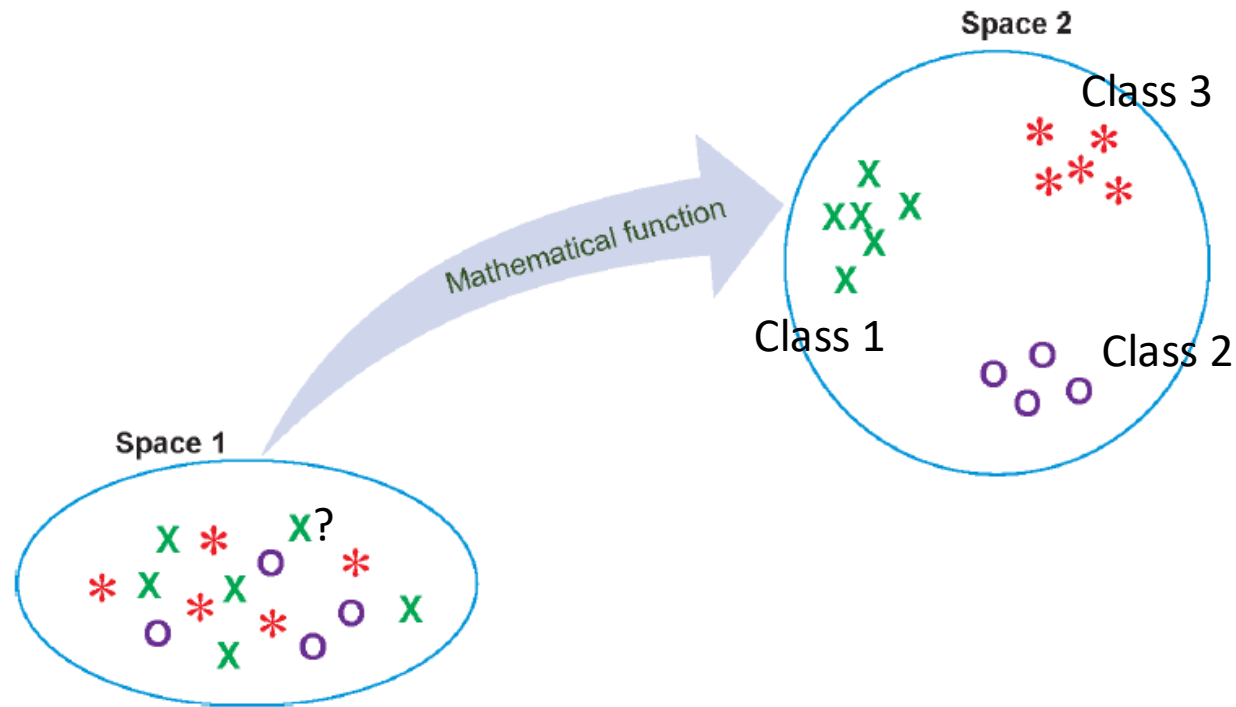
Mathematical View of Learning

- Learning process is indeed the process of finding parameters of an appropriate mathematical **mapping** by which an input vector is mapped to another space where similarities and dissimilarities of the “within-class” and “between class” data, are better projected, respectively.



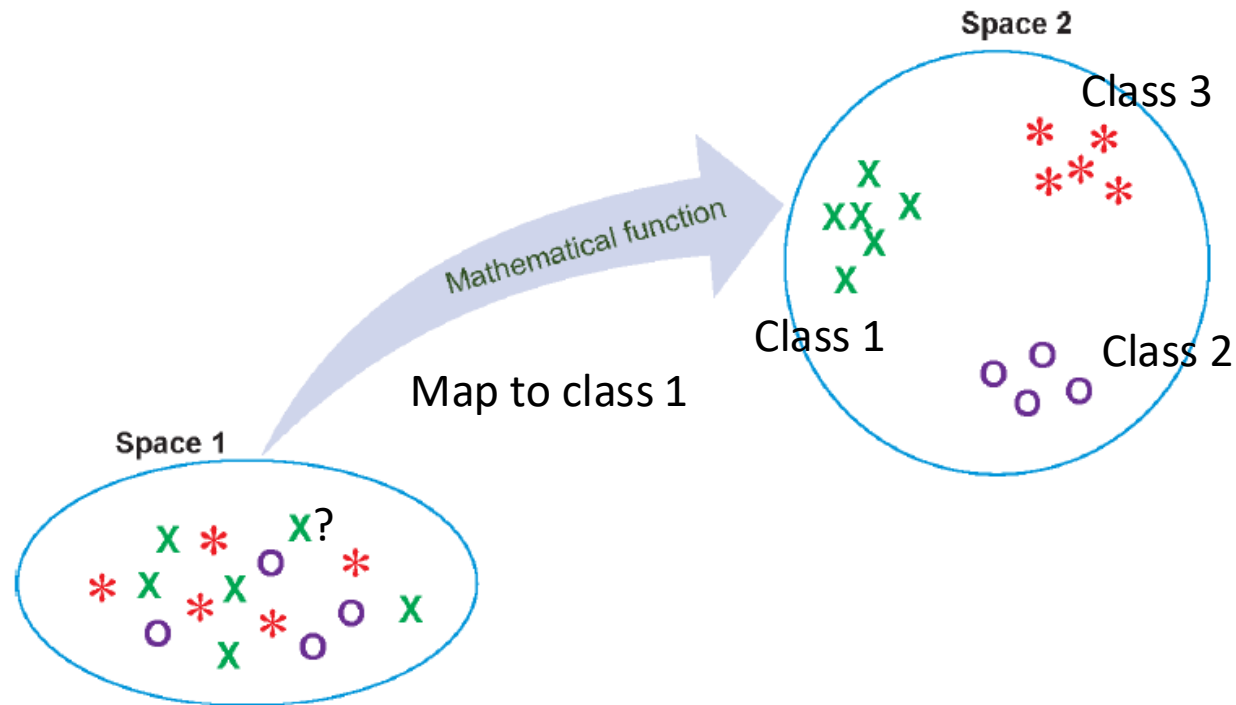
Mathematical View of Learning

- Learning process is indeed the process of finding parameters of an appropriate mathematical **mapping** by which an input vector is mapped to another space where similarities and dissimilarities of the “within-class” and “between class” data, are better projected, respectively.



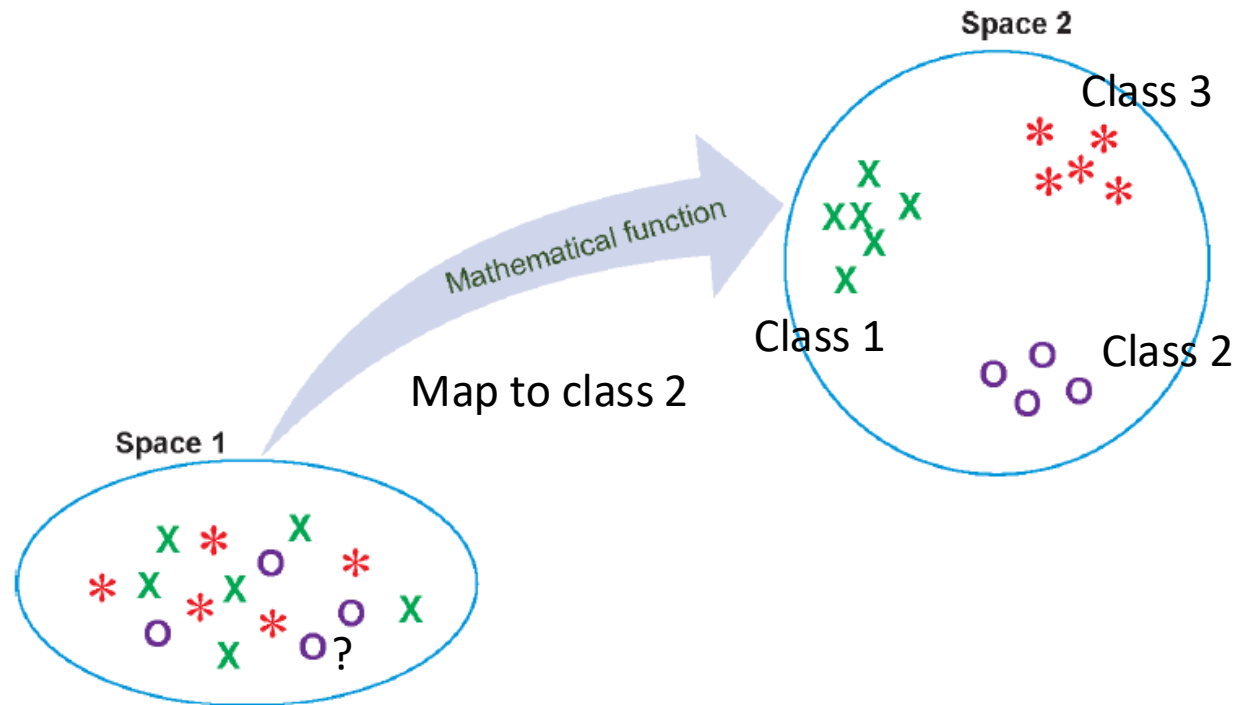
Mathematical View of Learning

- Learning process is indeed the process of finding parameters of an appropriate mathematical **mapping** by which an input vector is mapped to another space where similarities and dissimilarities of the “within-class” and “between class” data, are better projected, respectively.



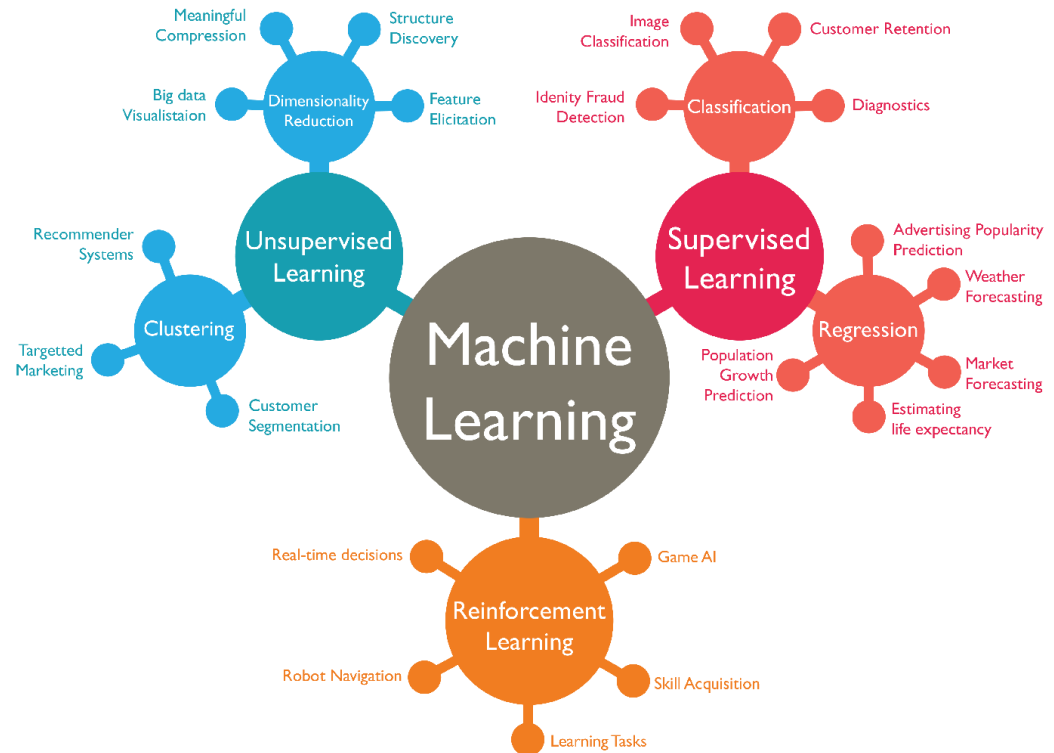
Mathematical View of Learning

- Learning process is indeed the process of finding parameters of an appropriate mathematical **mapping** by which an input vector is mapped to another space where similarities and dissimilarities of the “within-class” and “between class” data, are better projected, respectively.



Types of Machine Learning

- 1) Supervised learning
- 2) Unsupervised learning
- 3) Reinforcement learning



Types of Machine Learning

1) Supervised learning: Learning from labelled examples of correct behavior

Goal: To learn mapping from inputs to outputs, **given a labelled set of input-output pairs**

□ Categories of supervised learning:

- Classification – output is categorical (predicting a discrete class label).
- Regression – output is numerical (predicting a continuous value response variable).

□ Classification example: Image recognition

Task: given an image, predict the class label

- Training data: millions of labeled images
- Target: the image class



dog

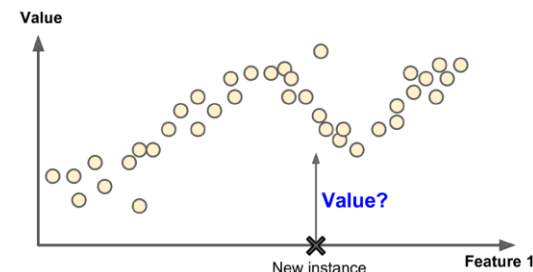


cat

□ Regression example: Car price prediction

Task: given a car, predict its price

- Training data: many examples of cars, including features (mileage, age, brand, etc.) and labels (price)
- Target: price of car



Types of Machine Learning

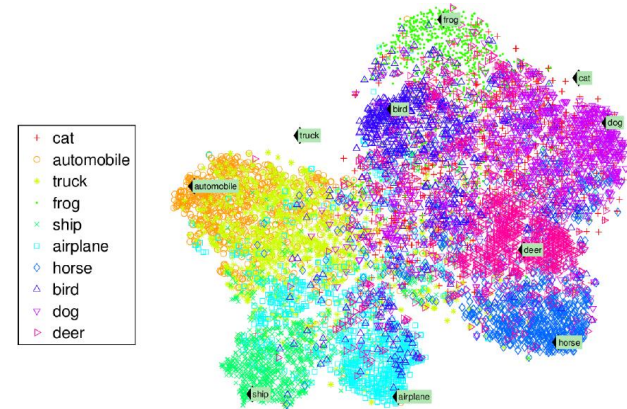
2) Unsupervised learning: no labelled examples available

Goal: To find “interesting patterns” in the data (knowledge discovery)

□ example: Clustering

Task: detect groups of similar objects

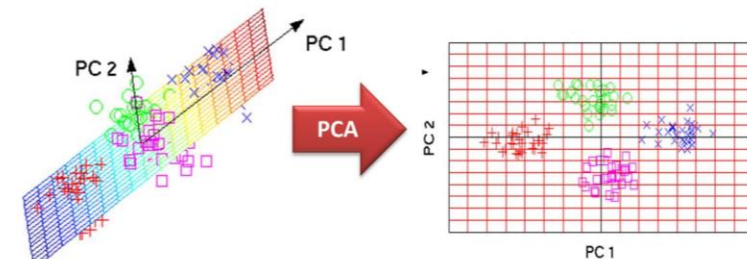
- Input: a lot of complex and unlabeled data



□ example: Dimensionality reduction

Task: Project high-dimensional data onto a low-dimensional space (to simplify, denoise, etc.)

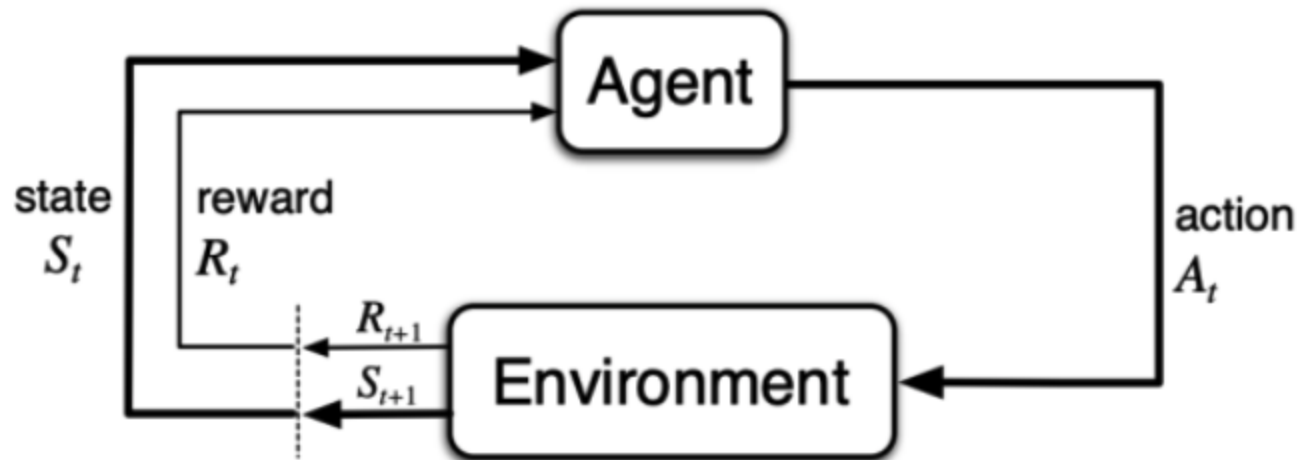
- Input: a large number of input features



Types of Machine Learning

- 3) Reinforcement learning: learning system (agent) observes the environment, selects, performs actions and receives a reward signal, tries to learn a strategy (policy) to maximize the positive reward signal

Goal: learn how to behave through interacting with an environment



Machine Learning Workflow

- ❑ Should I use ML on this problem?
 - Is there a pattern to detect? Can I solve it analytically? Do I have data?
- ❑ Gather and organize data.
 - Exploratory Data Analysis (EDA), cleaning, handling missing values etc.
 - Split data into two parts of training set (for training), and test set (for final evaluation after training)

Training set

test set

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
```

or more realistically into three parts of training set (for training), validation set (for evaluation during training), and test set (for final evaluation after training).

Training set

validation set

test set

```
# Splitting data into train (70%), validation (15%), and test (15%)
train_data, temp_data = train_test_split(data, test_size=0.3, random_state=42)
validation_data, test_data = train_test_split(temp_data, test_size=0.5, random_state=42)
```

- ❑ Choosing a model
 - derive a mathematical algorithm and translate it into code (e.g., python)
- ❑ Training the model (using training and validation sets)
 - Iterative optimization (loss/cost, optimization technique, regularization,...)
 - Evaluation (training and validation performance)
 - Hyperparameter tuning (using the validation set).
- ❑ Evaluate the trained model on the test set (unseen data during training) and improve model/dataset if necessary
- ❑ Save and lunch the trained model that had convincing test results

Machine Learning Workflow

❑ Should I use ML on this problem?

- Are predictions even possible? Can I solve it with an algorithm? Do I have data?

❑ Gather and organize data.

- Exploratory Data Analysis (EDA), cleaning, handling missing values etc.
- Split data into two parts of **training set**, and **test set**.

Training set

test set

- Or ideally into three parts of training set, **validation set** (for evaluation during training), and test set (for final evaluation after training).

Training set

validation set

test set

❑ Training the model (using training and validation sets)

- Iterative optimization (loss/cost, optimization technique, regularization,...).
- Hyperparameter tuning (using the validation set).
- Evaluation (training and validation performance).

❑ Evaluate the trained model on the test set (unseen data during training).

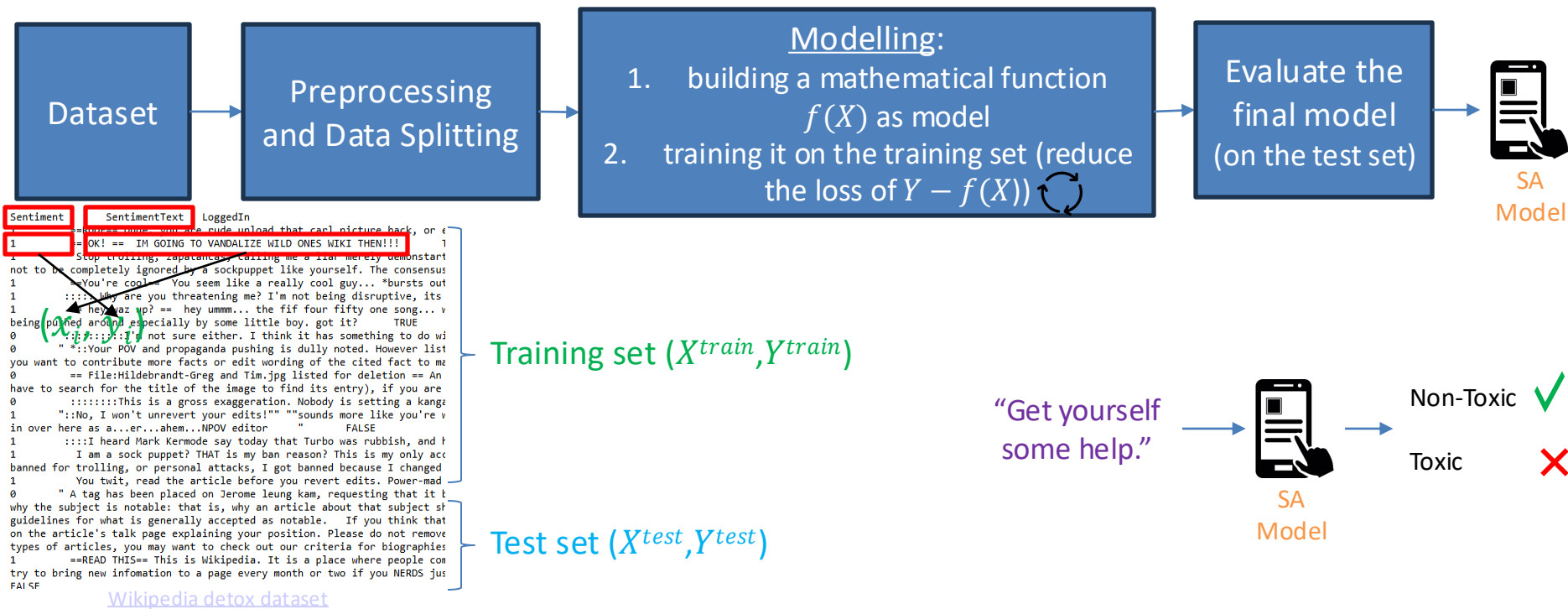
- **Test set** → held out until the very end, used once to **report the unbiased estimate** of generalization performance.

❑ Save and launch the trained model that had convincing test results.

What can be done with ML

□ Build a sentiment analysis (SA) model for website comments:

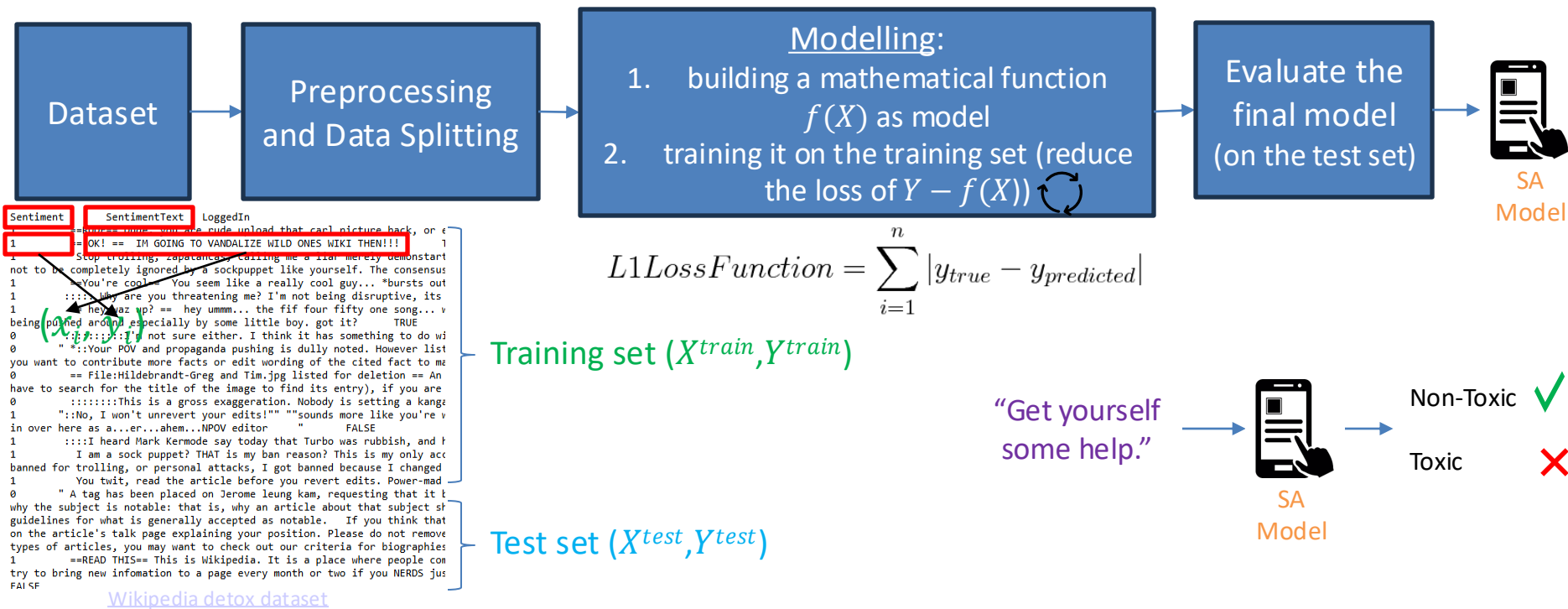
- Show some examples to the machine (labelled Dataset): different reviews left users on Wikipedia labelled as 0 (non-toxic) or 1 (toxic)
- EDA: exploratory data analysis (e.g., plotting distributions, correlations, etc.) and clean and preprocess if needed
- Split the dataset into training and test sub-sets
- Use engineering and domain knowledge to come up with a function that relates data inputs to their labels (X and Y)
- Train the model on the training set (reduce the loss between model output and true label in a recurring process)
- Test the final model on the test set after training
- If happy with performance, launch the final product (SA model)



What can be done with ML

□ Build a sentiment analysis (SA) model for website comments:

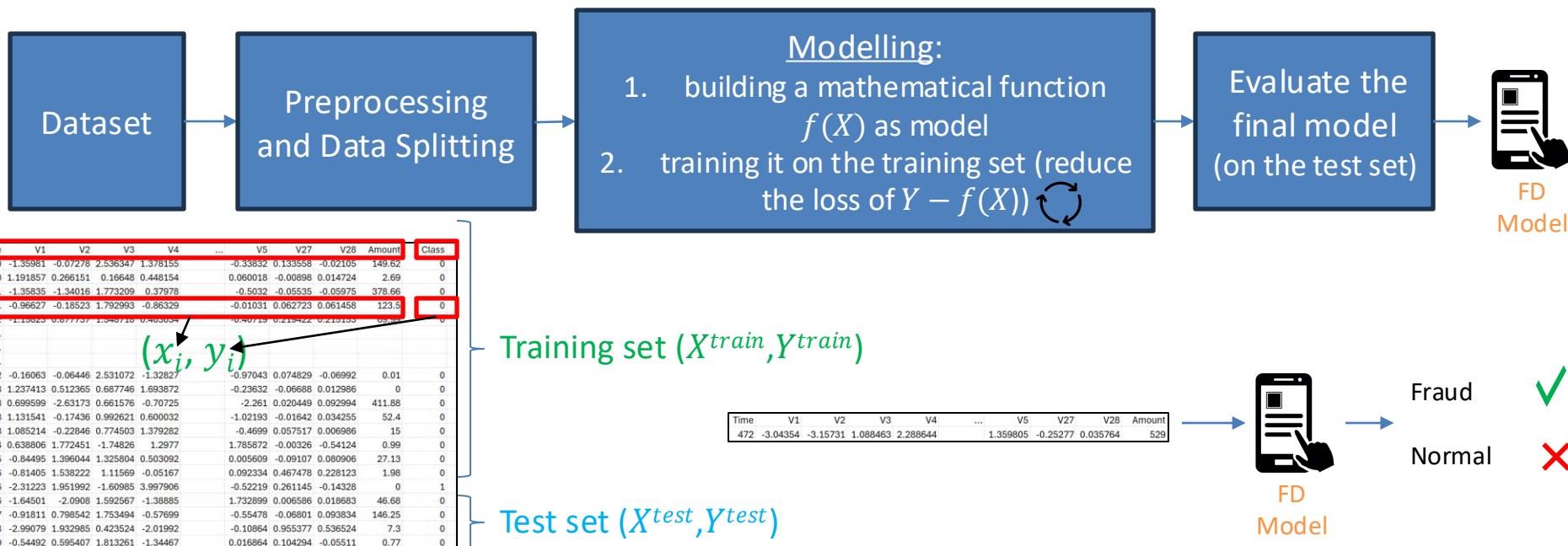
- Show some examples to the machine (labelled Dataset): different reviews left users on Wikipedia labelled as 0 (non-toxic) or 1 (toxic)
- EDA: exploratory data analysis (e.g., plotting distributions, correlations, etc.) and clean and preprocess if needed
- Split the dataset into training and test sub-sets
- Use engineering and domain knowledge to come up with a function that relates data inputs to their labels (X and Y)
- Train the model on the training set (reduce the loss between model output and true label in a recurring process)
- Test the final model on the test set after training
- If happy with performance, launch the final product (SA model)



What can be done with ML

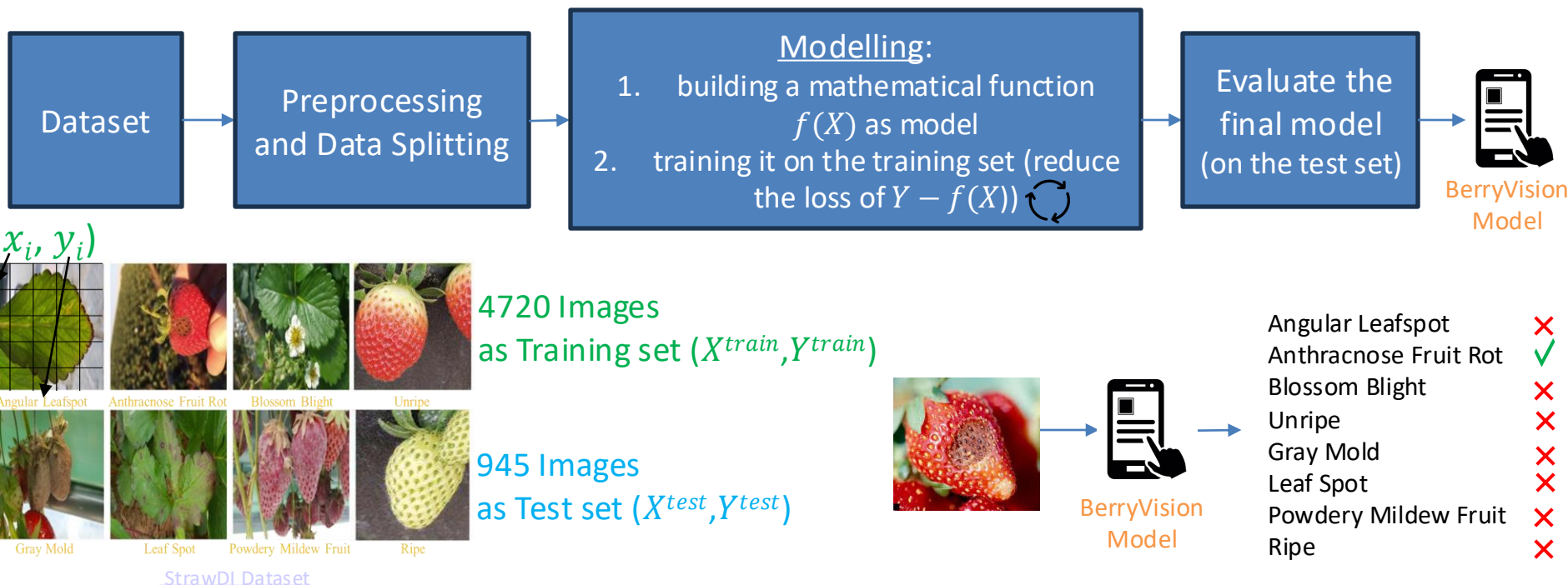


- ❑ Build a fraud detection (FD) model for credit card transactions :
 - Show some examples to the machine (labelled Dataset): transactions in two days made by European cardholders
 - Preprocess (e.g., cleaning, EDA, etc. if needed)
 - Split the dataset into training and test sub-sets
 - Use engineering and domain knowledge to come up with a function that relates data inputs to their labels (X and Y)
 - Train the model on the training set (reduce the loss between model output and true label in a recurring process)
 - Test the final model on the test set after training
 - If happy with performance, launch the final product (FD model)



What can be done with ML

- ❑ Build a BerryVision model for crop monitoring in a strawberry farm
 - Show some examples to the machine (labelled Dataset): sample images of ripe/unripe strawberries and different types of strawberry diseases
 - Preprocess (e.g., cleaning, EDA, etc. if needed)
 - Split the dataset into training and test sub-sets
 - Use engineering and domain knowledge to come up with a function that relates data inputs to their labels (X and Y)
 - Train the model on the training set (reduce the loss between model output and true label in a recurring process)
 - Test the final model on the test set after training
 - If happy with performance, launch the final product ([BerryVision](#) API)



Programming Language and IDE

❑ Python



- Python is a popular programming language to use in machine learning because it offers developers exceptional versatility and power while integrating with other software.
- This site has videos and course materials under [Lessons](#).
- Installing Python: <https://www.py4e.com/software-win.php>

❑ Jupyter Notebook



- Interactive computational/programming environment
- Installation guide: https://jupyterlab.readthedocs.io/en/stable/getting_started/installation.html
- Install it using conda (recommended)
- You may install Jupyter Notebook using the Anaconda Distribution (recommended for new users) <https://docs.jupyter.org/en/latest/index.html>
- More info: <https://www.datacamp.com/tutorial/tutorial-jupyter-notebook>

Machine Learning Libraries in this Course



- 1 Scikit-learn
Scikit-learn is known for its comprehensive collection of classical machine learning algorithms, making it a good choice for traditional tasks like classification and regression.



- 3 TensorFlow
TensorFlow is an open-source library widely used for numerical computation and large-scale machine learning, especially in deep learning.



- 2 PyTorch
PyTorch, another open-source library, is popular for its flexibility and dynamic computation graph, making it suitable for research and prototyping.

- 4 Others
There are numerous other libraries like Pandas (data analysis), NumPy (array processing), SciPy (optimization and linear algebra), Seaborn (statistical graphs), Matplotlib (visualization & plotting), ...

Setting Up Your Machine Learning Environment

1 — Prerequisites

Install Python 3.10.x and a package manager like pip.

2 — Install Libraries

Use pip to install necessary libraries:
TensorFlow, PyTorch, Scikit-learn, etc.

3 — Set Up Virtual Environments

Create virtual environments to isolate project dependencies and manage package versions.

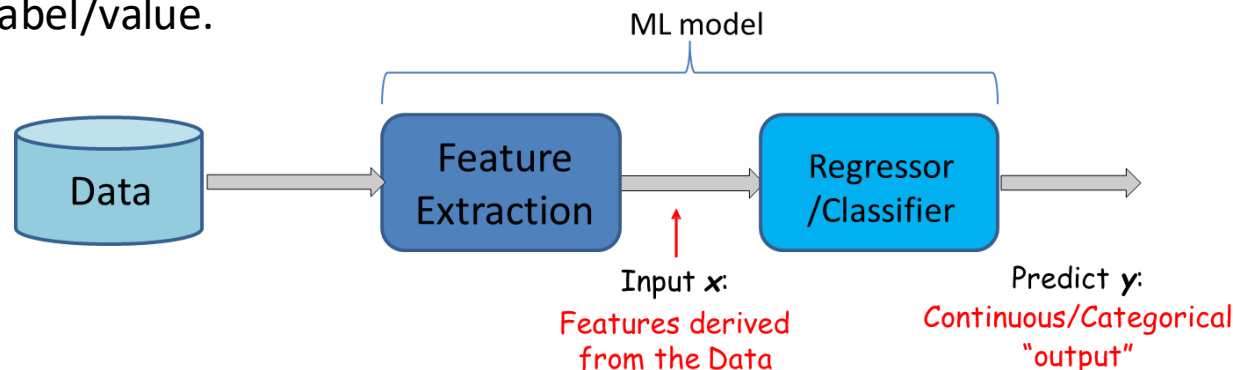


1.2: Regression:

Introduction to Simple Linear Regression

Supervised Learning (Regression/Classification)

- ❑ A statistical tool for predicting a continuous-valued response variable (Regression) or a categorical target variable (Classification)
 - Features/attributes and labelled classes/values are provided
 - Given an observation (a set of features), the goal is to predict that correct label/value.



❑ Notations:

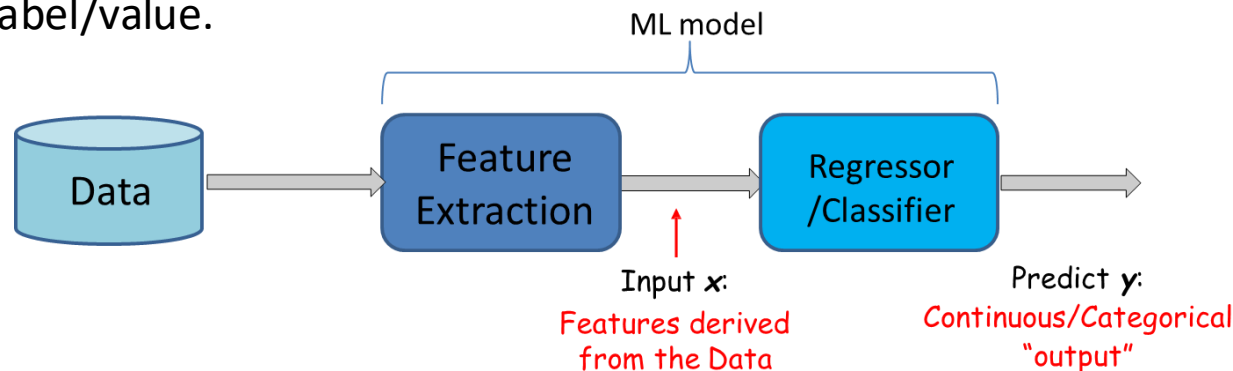
- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable $\rightarrow$ A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Prediction Examples:

- Electrical Power Demand
- Network Traffic in a Network
- Stock Price
- Email (Spam/not-Spam)
- Tumor (Benign/Malignant)

Supervised Learning (Regression/Classification)

- ❑ A statistical tool for predicting a continuous-valued response variable (Regression) or a categorical target variable (Classification)
 - Features/attributes and labelled classes/values are provided
 - Given an observation (a set of features), the goal is to predict that correct label/value.



❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable $\rightarrow$ A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Prediction Examples:

- Electrical Power Demand **(Regression)**
- Network Traffic in a Network **(Regression)**
- Stock Price **(Regression)**
- Email (Spam/not-Spam) **(Classification)**
- Tumor (Benign/Malignant) **(Classification)**

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable $\rightarrow$ A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (°C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable $\rightarrow$ A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (°C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable → A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (° C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable → A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (°C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable → A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair (x, y) → one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (°C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable → A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (°C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

i

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable → A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (° C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

$n=2$

Observations vs. Features

Features

- ❑ Variables/attributes used to describe an entity.
- ❑ Example: age, height, weight, blood pressure.

Observation (a.k.a. sample, instance, data point)

- ❑ A single row in your dataset: the values of *all features* for one entity.
- ❑ Example: (age=45, height=170cm, weight=70kg, blood pressure=120/80).

An observation = one example described by a set of feature values.

❑ Notations:

- m : denotes number of training examples/ number of observations
- x : denotes "input" variable (feature) - Temperature
 - Predictor variable → A change made in this variable affect the values of the output variable(s).
- y : denotes output/target variable/response variable that we are going to predict
- One pair $(x, y) \rightarrow$ one training example / one observation
- $(x^{(i)}, y^{(i)})$: denotes the i^{th} training example / i^{th} observation
- n : denotes the number of features

Temp (° C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..

$n=1$

Regression

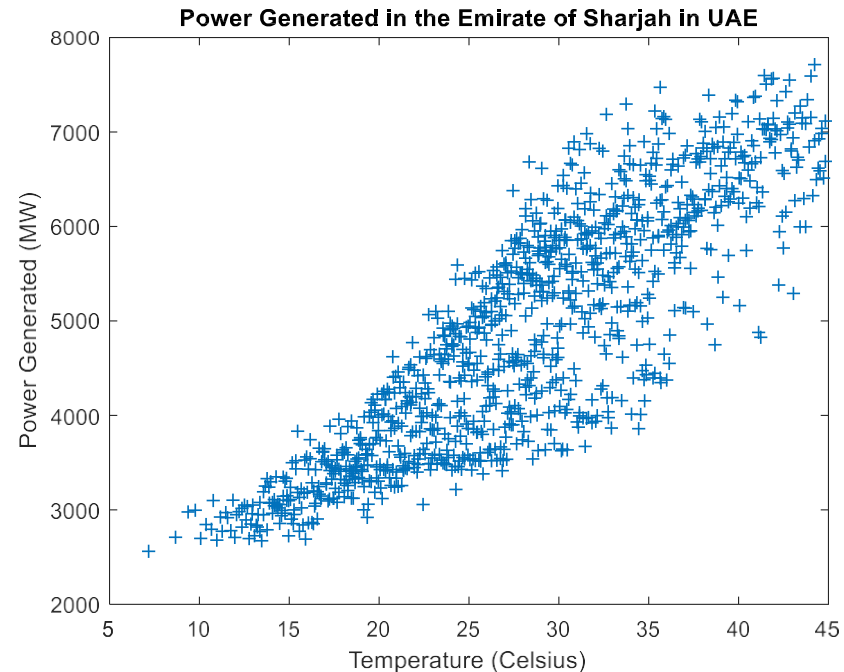
□ Example:

- Power Demand Prediction
- One way of predicting/estimating the power demand is to fit a straight line to this data
- Or we may find a better fit (*quadratic, exponential, etc.*)

□ Regression Problem:

- Output of the algorithm continuous valued number (power demand)

Temp (° C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..



Terminology:

given the labeled data

- Temperature (Celsius) -> **x: feature, covariate, or predictor**
- Power Demand (MWh) -> **y: label or response variable**

Regression

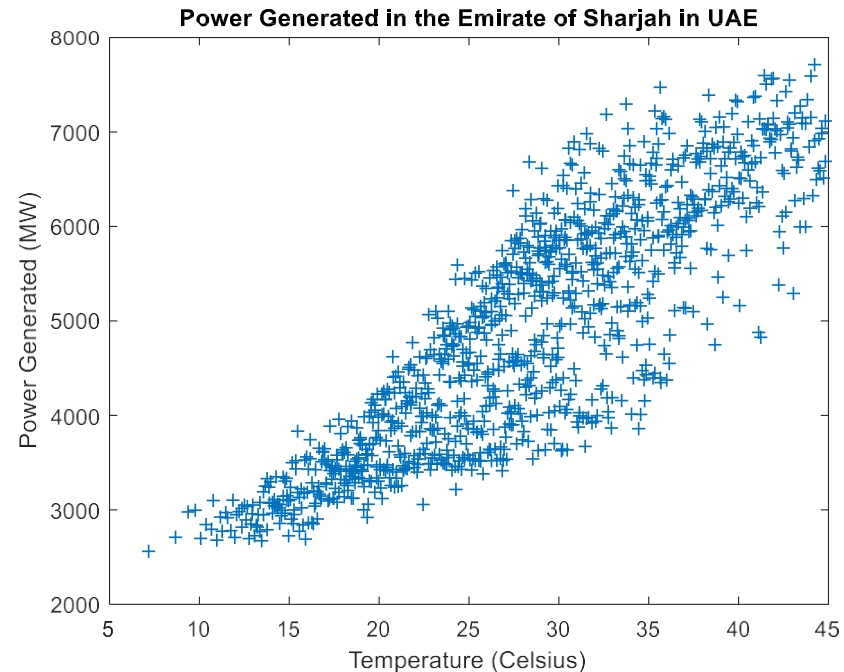
□ Example:

- Power Demand Prediction
- One way of predicting/estimating the power demand is to fit a straight line to this data
- Or we may find a better fit (*quadratic, exponential, etc.*)

□ Regression Problem:

- Output of the algorithm continuous valued number (power demand)

Temp (° C)	LOAD (Power Demand)
-24.2	18128
-10.9	19437
-5	14070
0	13955
3	14974
..	..
..	..



Terminology:

given the labeled data

- Temperature (Celsius) -> **x: feature, covariate, or predictor**
- Power Demand (MWh) -> **y: label or response variable**

Attributes/Features/Representations

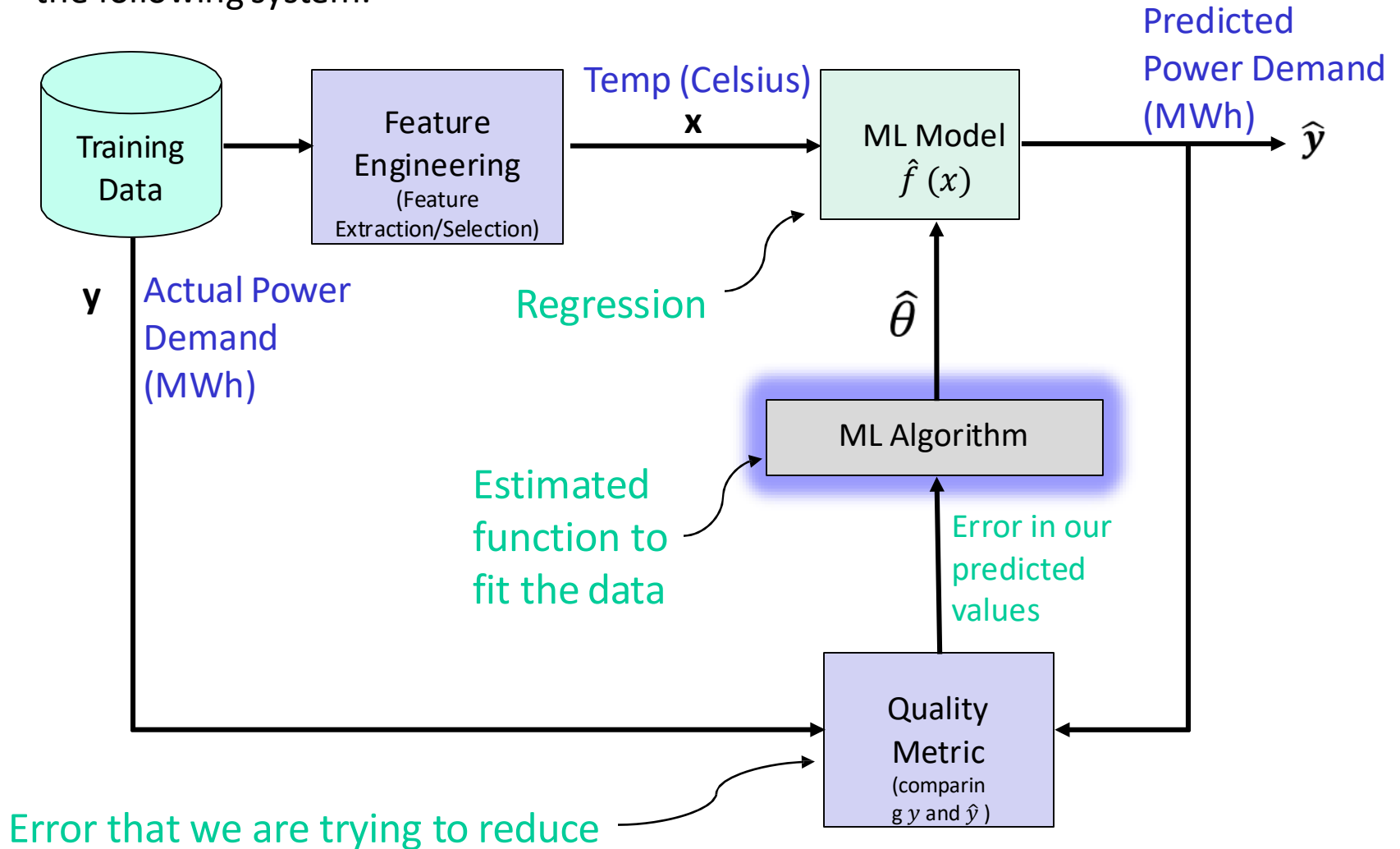
- ❑ In the example: We were looking at one “attribute” or “feature”:
 - Temperature

- ❑ Usually, we look at multiple features/input attributes. Example:
 - Temperature
 - Humidity
 - Date
 - Time
 - Holiday
 - House income
 - Type of customers: Residential / Commercial
 -

- ❑ Challenge: How to deal with large number of features?

Fitting the model

Using the prediction of power demand from temperature we have the following system:



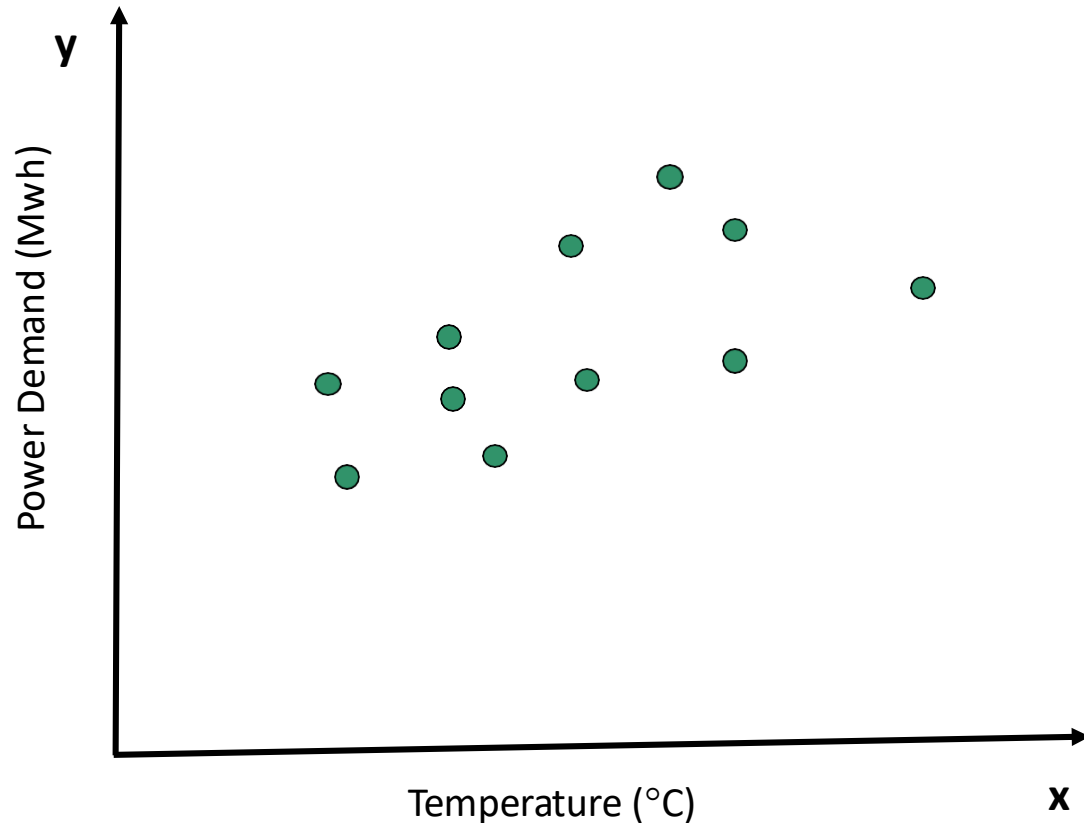
Θ vs. $\Theta^\wedge$

$\Theta^\wedge$ is an estimate of a parameter θ .

- θ = the *true but unknown* parameter of the data-generating distribution.
 - Example: the true regression coefficients in a linear model.
- $\theta^\wedge$ = the estimate of that parameter, computed from data (via maximum likelihood, least squares, Bayesian posterior mean, etc.).

Simple Linear Regression

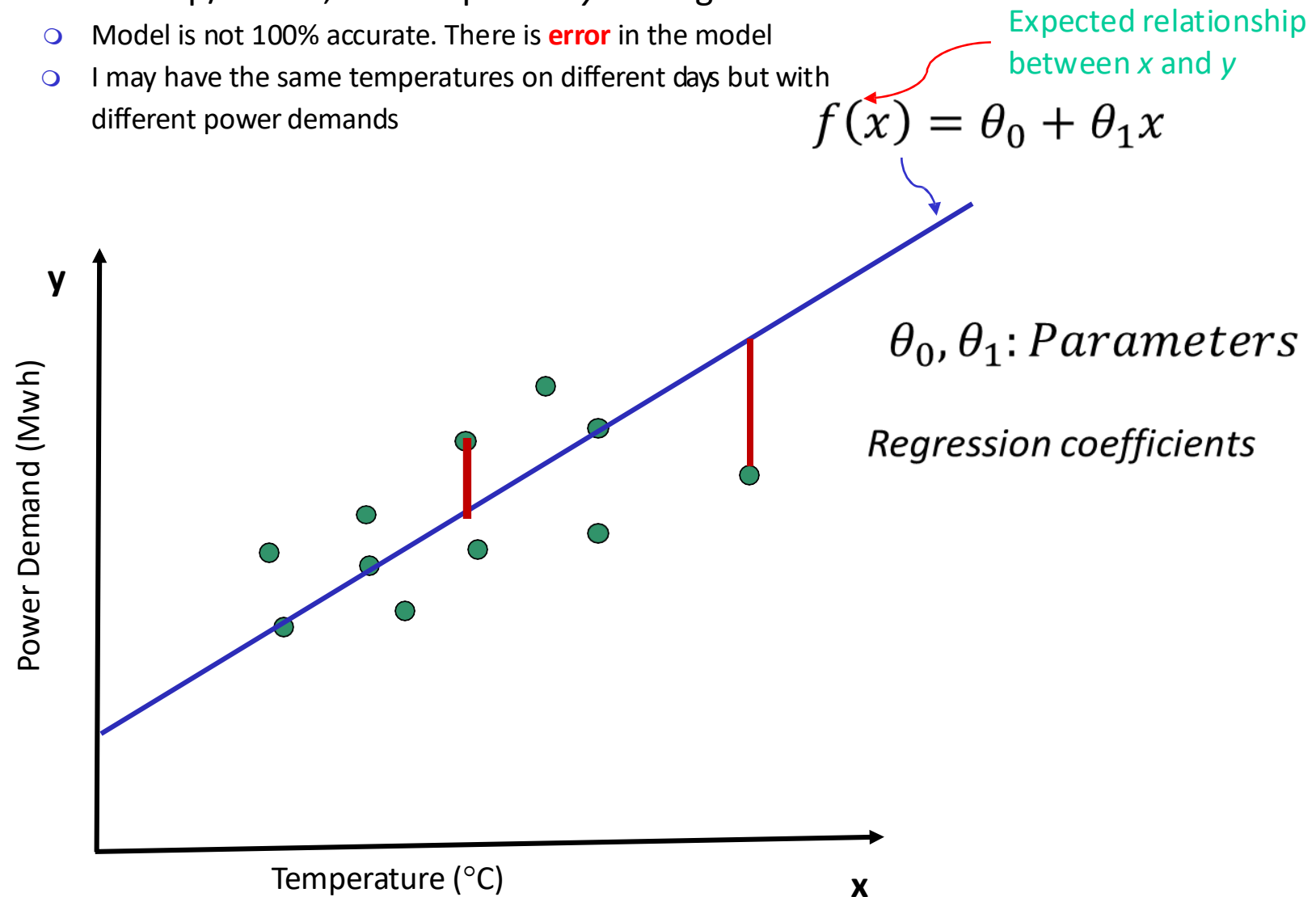
- **Model:** represents the expected relationship between x and y . Given the relationship/model, we can predict y if x is given.



Simple Linear Regression

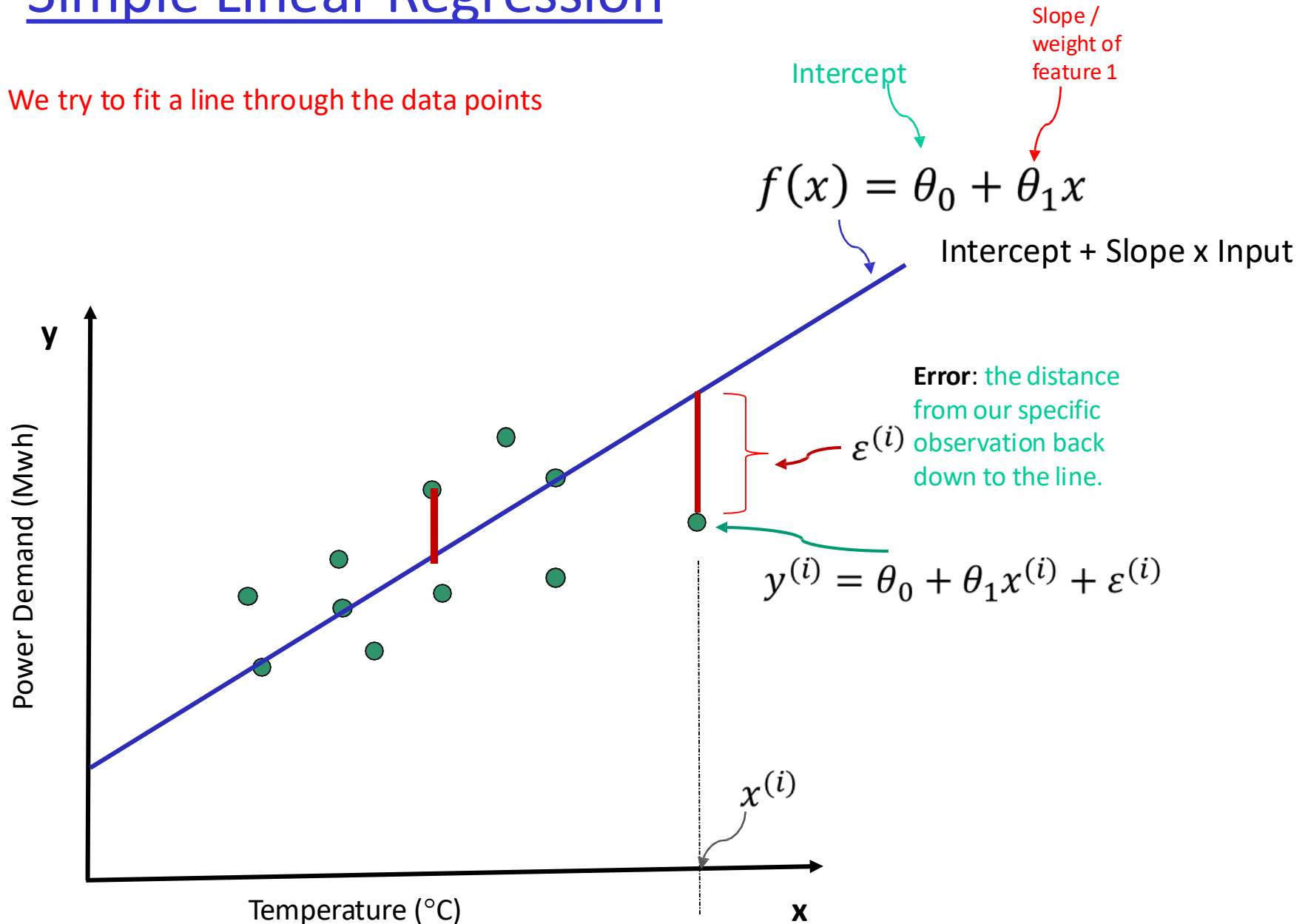
- **Model:** represents the expected relationship between x and y . Given the relationship/model, we can predict y if x is given

- Model is not 100% accurate. There is **error** in the model
- I may have the same temperatures on different days but with different power demands



Simple Linear Regression

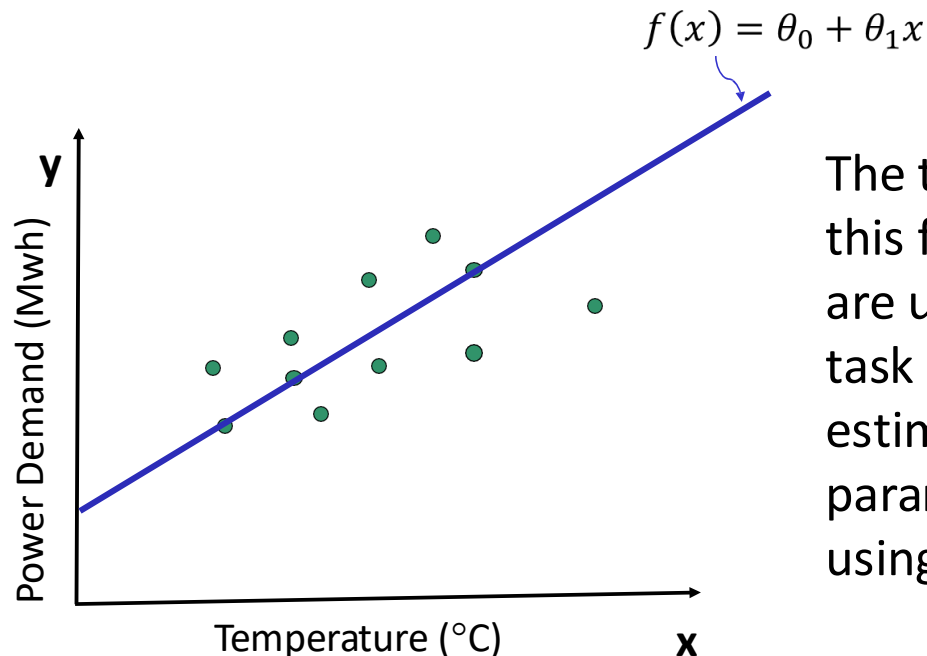
We try to fit a line through the data points



Θ vs. $\Theta^\wedge$

$\Theta^\wedge$ is an estimate of a parameter θ .

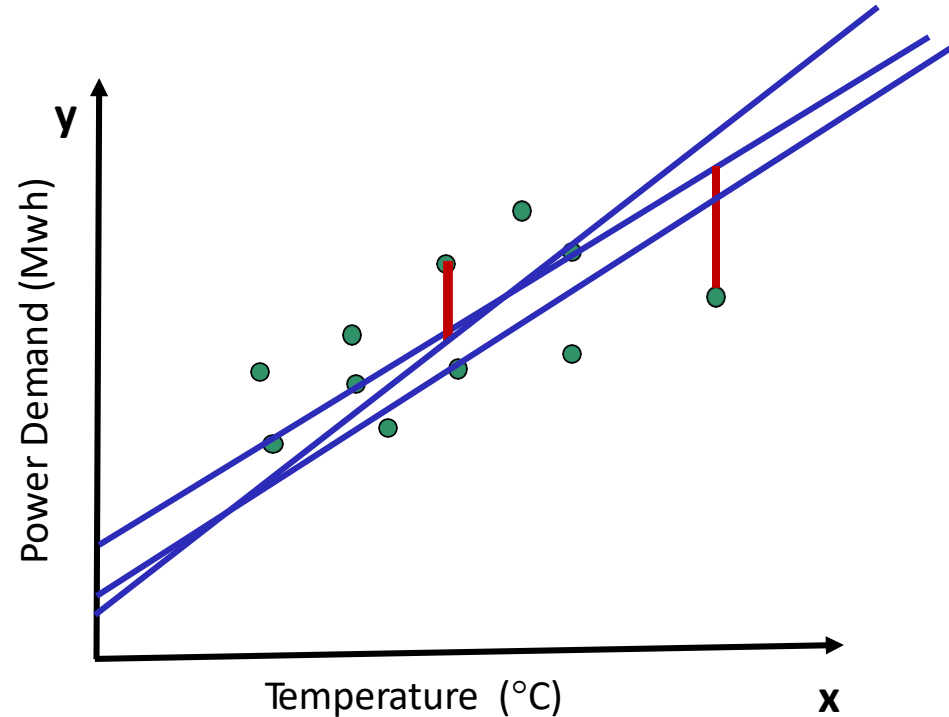
- θ = the *true but **unknown*** parameter of the data-generating distribution.
 - **Example: the true regression coefficients in a linear model.**
- $\theta^\wedge$ = the estimate of that parameter, computed from data (via maximum likelihood, least squares, Bayesian posterior mean, etc.).



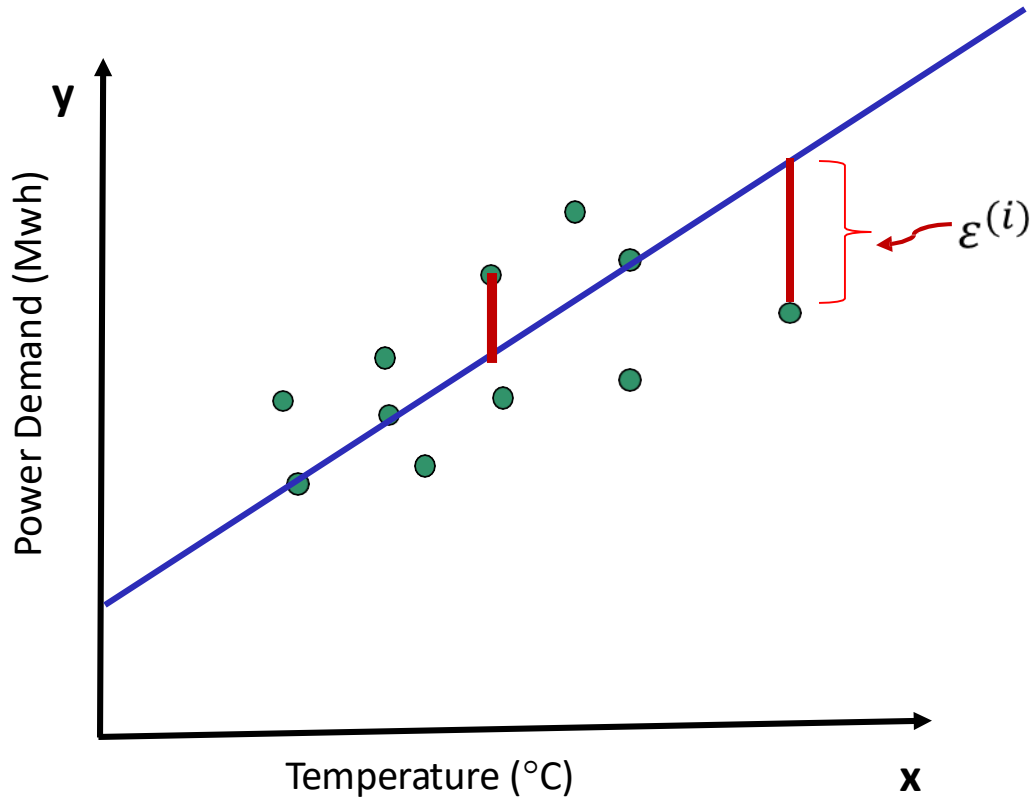
The true parameters of this function (θ_0 and θ_1) are unknown to us, so our task is to find a way to estimate them, i.e., find parameters $\theta^\wedge_0$ and $\theta^\wedge_1$ using our dataset.

Find the “best” line: How to measure the quality of the fit?

- ❑ Which line is the right line or the best fit to a given data set?
- ❑ Each one of these is given by a different set of parameters.
- ❑ We need to define a cost for every given line
- ❑ **Residual Sum of Squares (RSS)/sum of squared errors (SSE):**
Cost of each line will be based on RSS



Cost of using a specific line



Residual sum of squares (RSS)

$$\begin{aligned} RSS(\theta_0, \theta_1) = & [y^{(1)} - (\theta_0 + \theta_1 x^{(1)})]^2 \\ & + [y^{(2)} - (\theta_0 + \theta_1 x^{(2)})]^2 \\ & + \dots\dots\dots \\ & [y^{(m)} - (\theta_0 + \theta_1 x^{(m)})]^2 \end{aligned}$$

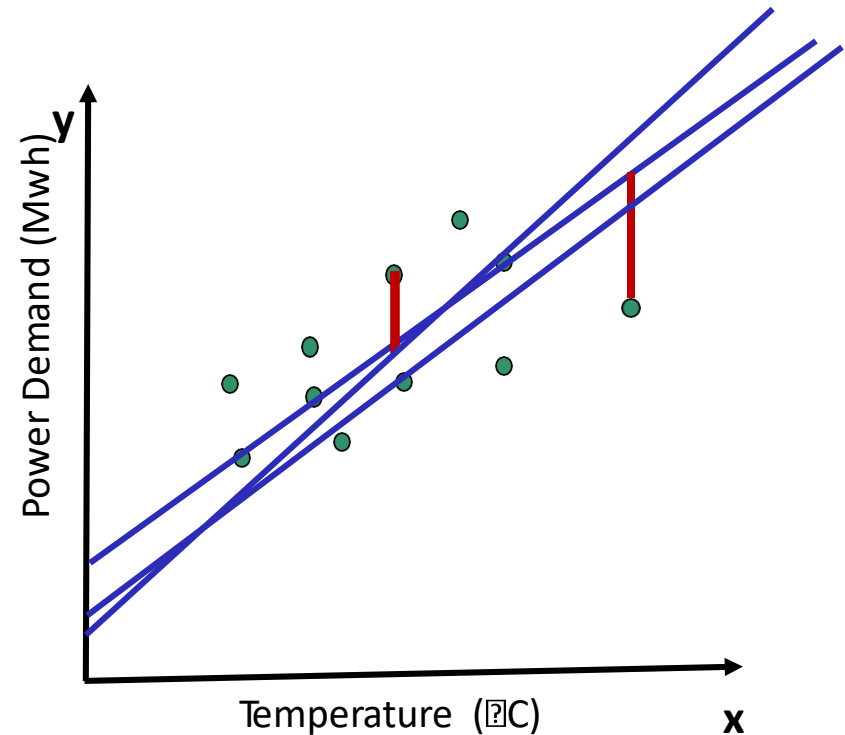
The square is to make sure all errors are positive and to penalize large errors more than small ones.

We are going to calculate the sum of the errors for all data points. Error is the difference between the estimated output and observed output. RSS is going to be function of θ_0 and θ_1

$$RSS(\theta_0, \theta_1) = \sum_{j=1}^m [y^{(j)} - (\theta_0 + \theta_1 x^{(j)})]^2$$

Find the “best” line

- We will try to find the best line according to the RSS.
- Ideally, we have to look at all possible lines, and we choose the one that minimizes the residual sum of squares (RSS).
- We are going to denote the resulting θ_0 and θ_1 as $\hat{\theta}_0$ and $\hat{\theta}_1$.
- There exist nice and efficient algorithms for computing this as $\hat{\theta}_0$ and $\hat{\theta}_1$.
- The algorithm searches over all possible θ s.



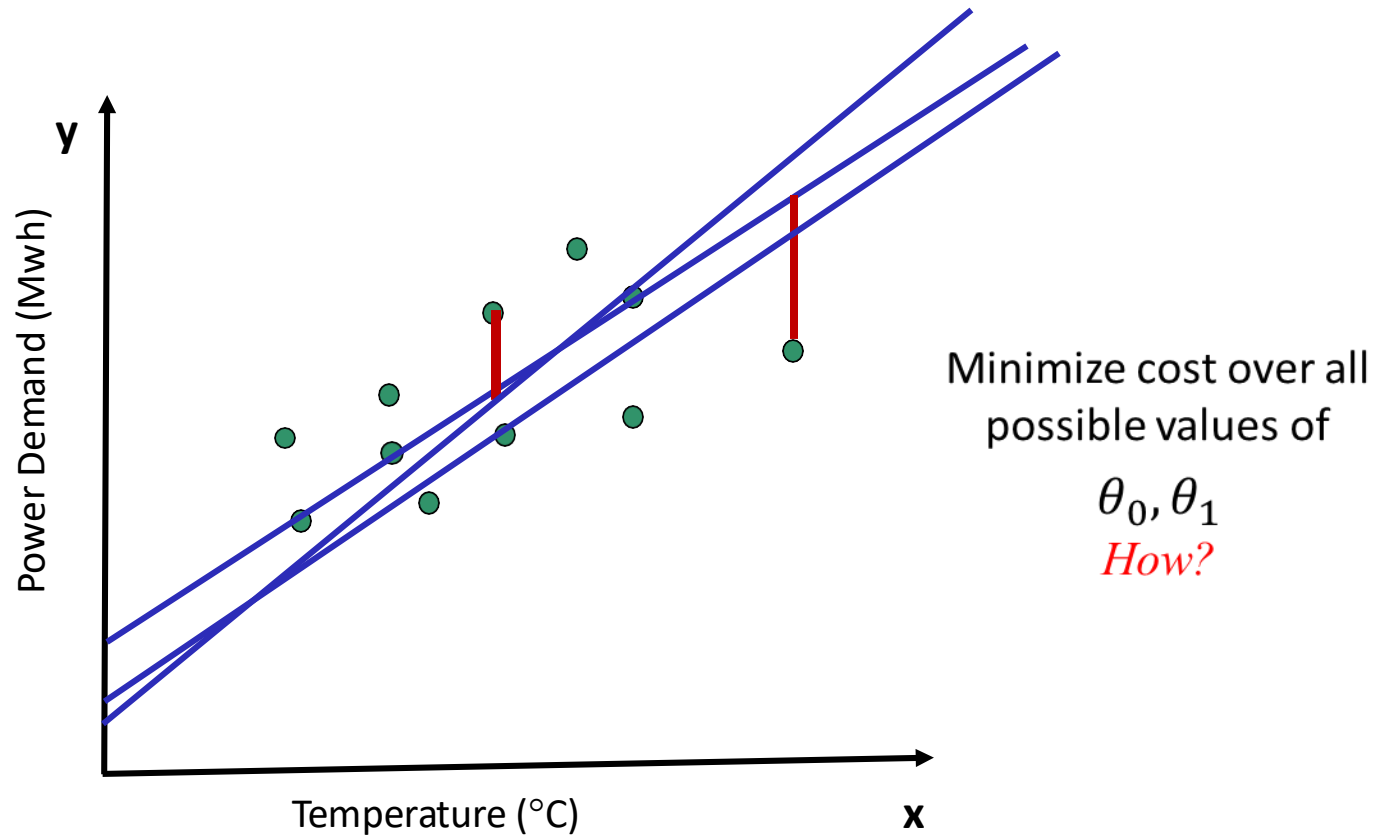
Residual sum of squares (RSS)

$$RSS(\theta_0, \theta_1) = \sum_{j=1}^m [y^{(j)} - (\theta_0 + \theta_1 x^{(j)})]^2$$

We want to iteratively search for the parameters θ_0 and θ_1 that **minimize** the RSS.

1.3: Find the Best Model: Optimization Techniques

Find the "best" line



Residual sum of squares (RSS)

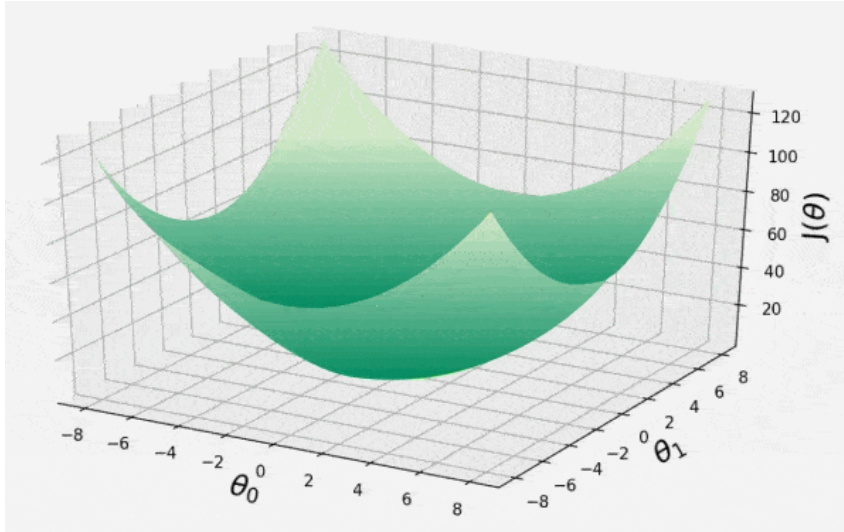
$$RSS(\theta_0, \theta_1) = \sum_{j=1}^m [y^{(j)} - (\theta_0 + \theta_1 x^{(j)})]^2$$

Minimizing the cost

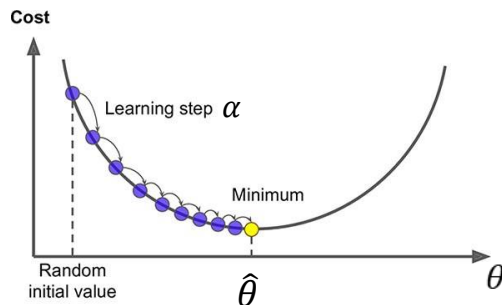
Minimize function over all possible θ_0 and θ_1

$$\min_{\theta_0, \theta_1} \sum_{j=1}^m [y^{(j)} - (\theta_0 + \theta_1 x^{(j)})]^2$$

$RSS(\theta_0, \theta_1)$ is a function of 2 variables $H(\theta_0, \theta_1)$



Optimization:

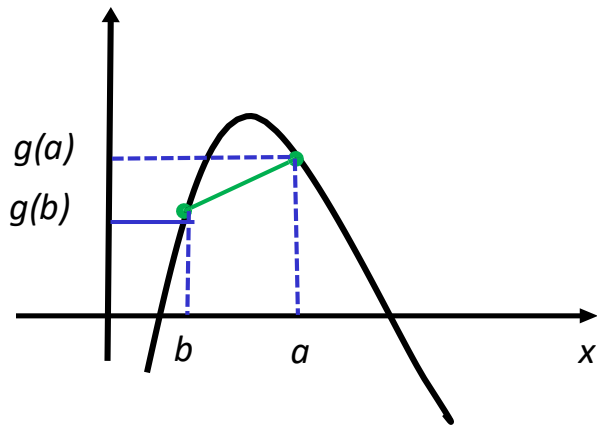


while not converged

$$\theta_{\text{iteration } t+1} \leftarrow \theta_{\text{iteration } t} - \alpha \frac{dg(\theta)}{d\theta} \Big|_{\theta_{\text{iteration } t}}$$

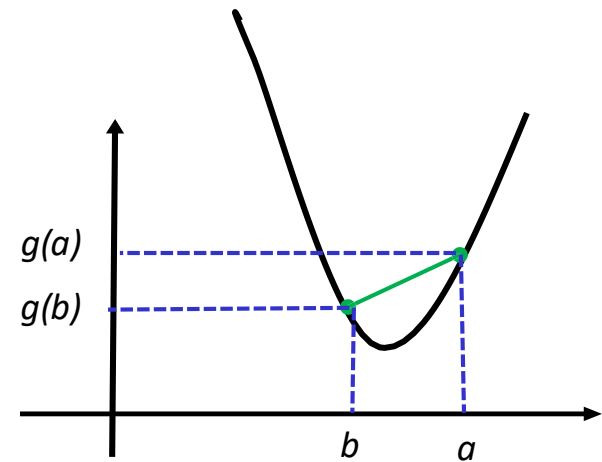
α is the step size (learning rate)

Optimization (Quick review)



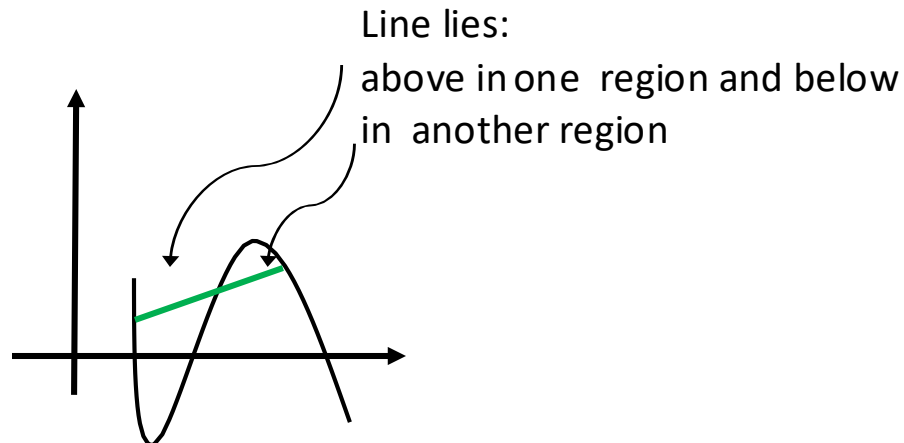
Concave: For any two values of a and b , the line between $g(a)$ and $g(b)$ (the green line above) lies below $g(x)$ everywhere.

It looks like you are in a cave (arch)



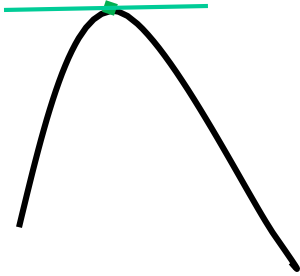
Convex: has the opposite property. This line lies above $g(x)$ everywhere.

Neither: The function does not satisfy either the concave or convex criteria.



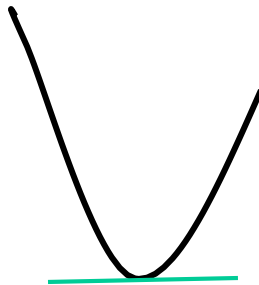
Finding the max or min analytically

Concave



- The maximum of the concave function
 $\max_x g(x)$
- The point where the derivative is 0.
There is no rate of change of this function at this point

Convex

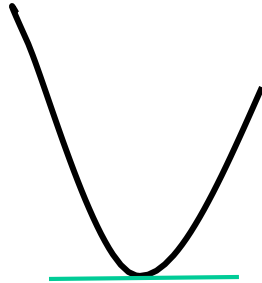


- The minimum of the convex function:
 $\min_x g(x)$
- The point where the derivative is 0.

- For the concave and convex functions, there is only one place where the derivative is equal zero.
- In contrast, for functions that are neither, we see that there are multiple locations where we have the derivative equaling 0 (global and local minimum/maximum)
- Thus, there are multiple solutions to the derivative is equal 0.

Optimization Process for RSS

Convex



- The minimum of the convex function:
 $\min_x g(x)$
- The point where the derivative is 0.

- ❑ **Write RSS** as a function of the parameters.
- ❑ **Take partial derivatives** w.r.t each parameter.
- ❑ **Set derivatives = 0** to find the “stationary point” (where slope is 0).
- ❑ **Solve the equations** for the parameter estimates (θ^0, θ^1) .
- ❑ The stationary point is a **minimum** because RSS is a **convex function** in linear regression.

Finding the max or min analytically – Ex.

- Assume we have the following function:

$$g(\theta) = 1 - (\theta - 5)^2$$

- Analytically, to find the max or min we must first take the derivative:

$$\frac{dg(\theta)}{d\theta} = \frac{d(1)}{d\theta} - \frac{d(\theta - 5)^2}{d\theta}$$

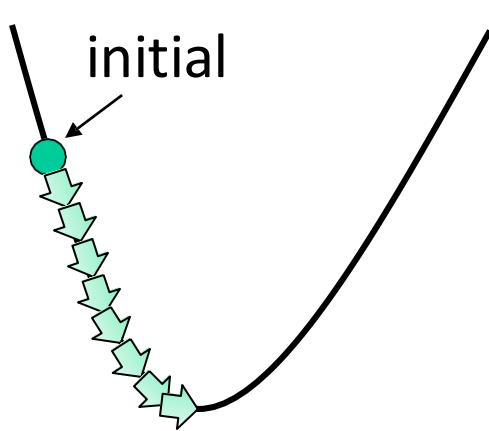
$$\frac{dg(\theta)}{d\theta} = \cancel{\frac{d(1)}{d\theta}}^0 - 2(\theta - 5) * 1$$

- Next, in order to determine where the rate of change is zero, we must set the first derivative to 0 and solve for θ

$$0 = -2(\theta - 5) * 1 \longrightarrow 0 = \underbrace{-2\theta}_{\text{green circle}} + 10 \longrightarrow \frac{2\theta}{2} = \frac{10}{2} \longrightarrow \theta = 5$$

Choosing The Learning Rate - Stepsize

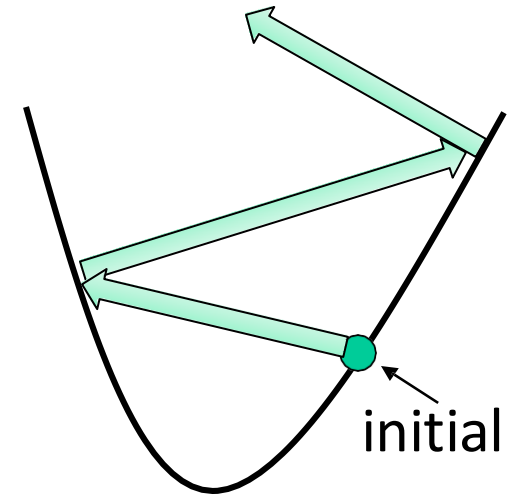
A static learning rate is one which remains constant during all iterations.



too small

A learning rate which is too small requires many iterations before reaching the minimum.

When selecting a static learning rate, care must be taken to ensure that the learning rate is appropriate



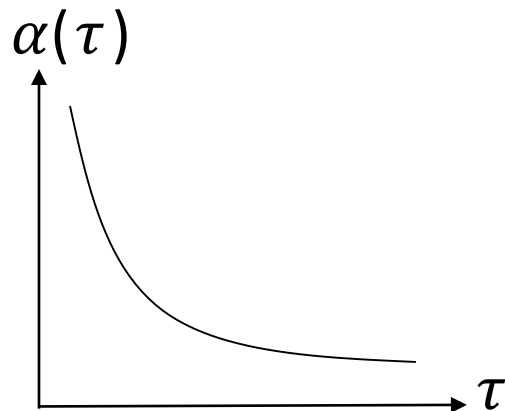
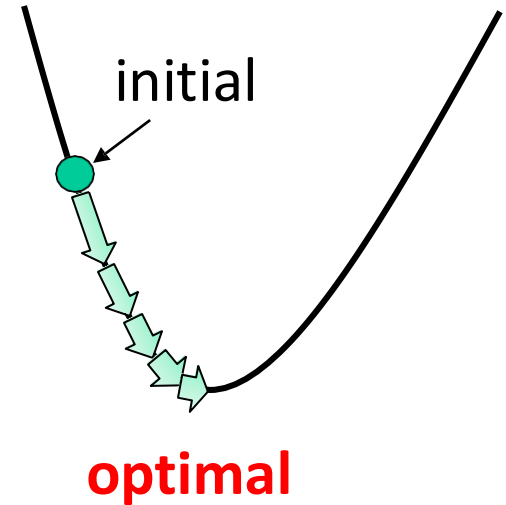
too large

A learning rate which is too large leads to severe updates and results in divergence.

Choosing The Learning Rate - Stepsize

Decreasing learning rate

- A better practice when selecting a learning rate is the idea of a dynamic (decreasing) learning rate which decreases over time. This is ideal because it allows the algorithm to swiftly identify the minimum point.
- The most basic decreasing learning rate schedule is:



$$\alpha(\tau) = \frac{\alpha}{\tau}$$

initial learning rate

iteration number

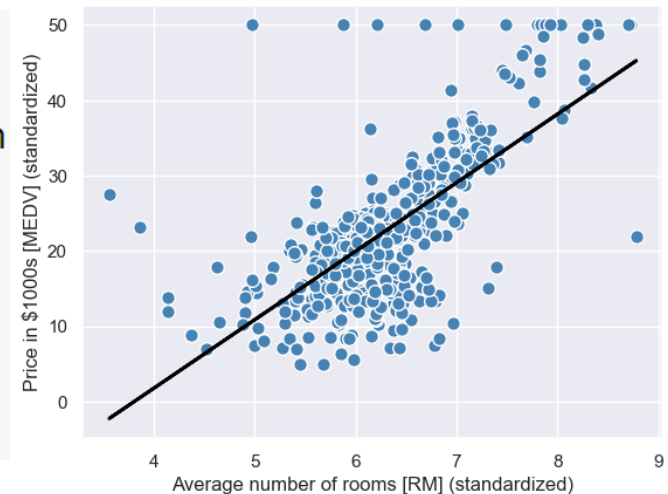
Example:

❑ Housing dataset from UCI repository:

<https://raw.githubusercontent.com/rasbt/python-machine-learning-book-2nd-edition/master/code/ch10/housing.data.txt>.

- ❑ Features: CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT
- ❑ Target variable: MEDV (Median value in \$1000s)
- ❑ To fit a linear regression model, we are interested in those features that have a high correlation and linear relationship with our target variable MEDV.

```
X = df[['RM']].values
y = df['MEDV'].values
from sklearn.linear_model import LinearRegression
slr = LinearRegression()
slr.fit(X, y)
print('Slope: %.3f' % slr.coef_[0])
Slope: 9.102
print('Intercept: %.3f' % slr.intercept_)
Intercept: -34.671
```



Example:

- We want to split our dataset into separate training (to fit the model) and test datasets (to evaluate model's performance to generalize to unseen data).

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=0)
```

```
def RSS(y, y_hat):
    total_RSS = 0.
    for y_, y_hat_ in zip(y, y_hat):
        total_RSS += (y_ - y_hat_)**2
    return total_RSS
```

```
from sklearn.linear_model import LinearRegression
SLR = LinearRegression() # Simple linear regression (SLR) model
SLR.fit(X_train, y_train)

# the estimated parameters
lin_reg.intercept_, lin_reg.coef_

# make prediction on the test set
y_predict_reg = SLR.predict(X_test)

# print the RSS value
print(RSS(y_test, y_predict_reg))
```